

AML Compliance Platform

Architecture and API Flow Guide

Project: aml-code

Audience: project presentation and technical walkthrough

Implementation reviewed: backend branch, August 2026

1. What the system does

This project is an Anti-Money Laundering compliance backend. It accepts a financial transaction, builds behavioral history features for the sender and receiver, asks a FastAPI risk-scoring service for an assessment, stores the assessment, and opens an investigation case when the model says the transaction should be flagged.

The platform also provides CRUD APIs for customers, accounts, and rules, plus case review and compliance-report APIs. A gateway presents one client-facing entry point, while Eureka provides service discovery.

2. Runtime topology

Client or frontend

```
|  
v  
API Gateway :8080  
|-- /api/v1/auth/** -----> Auth Service :8082  
|-- transaction, account,  
| customer, case, rule,  
| and report APIs -----> Transaction Service :8081  
|  
+-----> Eureka Registry :8761 (discovery)
```

Transaction Service :8081

```
-- Oracle database: localhost:1521/freepdb1, schema capstone  
-- FastAPI risk model: http://localhost:8500  
+-- shared DTOs, entities, repositories, and EventBus
```

The gateway has explicit local routes, so local requests do not depend on Eureka being available. All three Spring services are also configured as Eureka clients; Eureka itself runs as the registry server.

3. Maven modules

aml-common

The shared contract and persistence module. It contains request and response DTOs, JPA entities, repositories, enums, and FastAPI request/response DTOs. This keeps the modules aligned on the data shape exchanged between layers.

aml-config

Shared Spring configuration. It provides security/JWT-related configuration, datasource support, HTTP client configuration, Jackson configuration, and the in-memory EventBus.

aml-eureka-server

The service registry. It runs on port 8761 and uses `@EnableEurekaServer` so services can register and discover one another.

aml-gateway

The reactive API gateway on port 8080. It handles CORS and forwards client paths to the

auth or transaction service. It also carries the JWT secret configuration used at the edge.

aml-transaction-service

The main business service on port 8081. One service currently owns transactions, accounts, customers, rules, investigation cases, risk assessments, and generated reports.

aml-auth-service

The authentication service on port 8082. It authenticates credentials and returns a signed JWT with the username, role, token type, and expiration.

4. Authentication flow

1. The client sends POST /api/v1/auth/login to the gateway.
2. The gateway forwards the request to Auth Service :8082.
3. AuthController passes the username and password to Spring AuthenticationManager.
4. On success, JwtTokenService signs a token using JWT_SECRET or the development default.
5. The client sends Authorization: Bearer <token> on protected API calls.

Default development credentials are admin / admin123 unless environment variables override them. Do not use those values outside local development.

5. Main transaction flow

POST /api/v1/transactions

```
|
v
1. Validate request and load sender and receiver accounts
|
2. Update their bank locations from the request
|
3. Save the transaction with status COMPLETED
|
4. Calculate sender and receiver transaction history
|
5. Save TransactionFeatureSnapshot
|
6. Build FastApiRequest and call POST <fastapi.base-url>/predict
|
7. Save RiskAssessment from FastApiResponse
|
8. If shouldFlag=true: set transaction UNDER_REVIEW and create a case
|
9. Publish the mapped transaction on the in-memory EventBus
|
v
201 response containing TransactionResponse
```

The FastAPI response supplies requestId, riskScore, riskCategory, modelConfidence, shouldFlag, decisionThreshold, ensembleMethod, explanation, recommendation, historyContext, and processingTimeMs. The Java service stores these values so the later case and report views have the decision context.

Important behavior: the risk model call is part of the create-transaction request. A

model-service failure can therefore prevent the transaction request from completing, depending on the client/error handling configuration.

6. Flagged transaction to investigation case

The case is created directly by InvestigationCaseService after the risk assessment is saved. It is not currently created by a separate case microservice or an event subscriber.

The service:

- Prevents duplicate cases for the same business transaction ID.
- Requires a stored risk assessment with shouldFlag=true.
- Sets the transaction to UNDER_REVIEW.
- Defaults priority to CRITICAL for a CRITICAL risk category, otherwise HIGH.
- Derives case type: SANCTIONS, STRUCTURING, TRADE_BASED, or GENERAL.
- Routes SANCTIONS to SANCTIONS_EXPERT, STRUCTURING to BEHAVIORAL_EXPERT, CRITICAL cases to SENIOR_COMPLIANCE, and other cases to COMPLIANCE_LEAD.
- Starts the case at PENDING.

Case resolution accepts APPROVED, REJECTED, ESCALATED, or MANUAL_REVIEW. A resolved case sets its transaction back to COMPLETED; an escalated case remains ESCALATED.

7. Report flow

1. POST /api/v1/reports/generate with a case ID.
2. ReportGenerationService loads the case, its transaction, and the latest risk assessment.
3. The current implementation accepts the TEMPLATE provider only.
4. It builds a compliance review report from stored transaction, case, and risk values.
5. The report is stored and its ID is written back to the case.
6. GET /api/v1/reports/{id} returns the report DTO.
7. GET /api/v1/reports/{id}/download returns the stored content as text/plain.

The request model contains provider and format fields for future providers and formats, but non-TEMPLATE providers currently fail with a configuration error. The download endpoint currently returns TXT content, even though the broader specification discusses PDF reports.

8. API map through the gateway

Auth Service :8082

POST /api/v1/auth/login

Transaction Service :8081

Transactions:

GET /api/v1/transactions

POST /api/v1/transactions

GET /api/v1/transactions/{id}

GET /api/v1/transactions/business/{transactionId}

PUT /api/v1/transactions/business/{transactionId}/review

GET /api/v1/transactions/account/{accountId}

GET /api/v1/transactions/{transactionId}/risk

Accounts:

GET, POST /api/v1/accounts

GET, PUT, DELETE /api/v1/accounts/{id}

Customers:

GET, POST /api/v1/customers

GET, PUT, DELETE /api/v1/customers/{id}

Cases:

GET, POST /api/v1/cases

GET, PUT /api/v1/cases/{id}

POST /api/v1/cases/{id}/resolve

Rules:

GET, POST /api/v1/rules

PUT, DELETE /api/v1/rules/{id}

Reports:

POST /api/v1/reports/generate

GET /api/v1/reports/{id}

GET /api/v1/reports/{id}/download

For direct local testing, replace the gateway base URL with `http://localhost:8081` for transaction-service endpoints, or `http://localhost:8082` for login.

9. Data and responsibility boundaries

Customer and Account provide the parties and account context. Transaction records the payment. TransactionFeatureSnapshot stores derived history statistics. RiskAssessment stores the model decision and explanation. InvestigationCase stores the human-review workflow. GeneratedReport stores the rendered compliance output.

The transaction service exposes DTOs from its controllers rather than returning JPA entities directly. Account holders are represented through application IDs in the model; review the database configuration before presenting this as a complete no-PII guarantee.

10. Event-driven status: implemented versus planned

Implemented:

- aml-config contains an in-memory EventBus.
- TransactionService publishes a TransactionResponse after transaction processing.

Current synchronous path:

- TransactionService directly calls InvestigationCaseService.
- ReportGenerationService is invoked by the reports controller.
- No separate notification service is present in the Maven module list.
- No Kafka or external event broker is configured.

The older complete specification describes a larger asynchronous chain of TransactionFlaggedEvent, CaseCreatedEvent, ReportGeneratedEvent, and NotificationSentEvent. That is a useful target architecture, but it should be presented as planned or future work, not as the current runtime behavior.

11. Local startup order

Prerequisites: Java 17, Maven, Oracle database configured for the application, and the FastAPI model service on port 8500.

From code/aml-system:

1. mvn clean install
2. Start aml-eureka-server on 8761.
3. Start aml-auth-service on 8082.
4. Start aml-transaction-service on 8081.
5. Start aml-gateway on 8080.
6. Send login and business API requests through <http://localhost:8080>.

The transaction service uses ddl-auto: create-drop, so database schema lifecycle should be checked before using this configuration with persistent data.

12. Presentation script

Start at the gateway: one client entry point and two explicit downstream destinations. Show login next: credentials become a JWT, which is sent as a Bearer token. Then follow one transaction request through validation, history features, FastAPI prediction, risk persistence, and automatic case creation. Finish with case resolution and report generation. Be explicit that the current implementation is synchronous at the case boundary and that the broader event-driven notification chain is future architecture.